HOLY ANGEL UNIVERSITY

Name: Ong, Sean Keiffer S.
Section: CS-302

Date: September 20, 2026
Course: Computer Science

Final Project Proposal (Revised)

App Name: ligHAU

1. **Purpose & Audience**
    a. **Purpose**
    - Students still struggle to navigate Holy Angel University's large campus and locate classrooms, restrooms, parking areas, dining spaces, offices, and other facilities. I experienced this most as a freshman during enrollment, and even now, there are still areas of the campus I am unfamiliar with. Although physical campus maps are available, they are limited to certain locations. Students have the right to access and make use of campus facilities, and this app further supports that access by providing convenient and updated information about the university's amenities, facilities, and locations.

    b. **Audience**
    - **Primary Users:** Application administrators and maintainers who manage campus locations, facility information, and map updates.
    - **Secondary Users:** Students, visitors, and other stakeholders who need a more accessible and informative way to navigate Holy Angel University, as the current static tarpaulin map provides limited details and makes it difficult to visualize buildings, classrooms, restrooms, and other campus facilities.

2. **Section/Routes**
    a. **Login Screen**
    - Exists to let users sign in, continue as guests, or access the system as an administrator.

    b. **Campus Map Screen**

- It serves as the main interface where users can explore Holy Angel University and locate buildings, classrooms, offices, restrooms, parking spaces, and other facilities.

c. Admin Dashboard

- Exists to let administrators add, edit, and remove campus buildings, rooms, facilities, and chatbot knowledge stored in Supabase.

3. Core Features/MVP

#	Feature	Still in the MVP?	Flutter pieces it needs	Honest Estimate
1	**Interactive 2D Map**	Keep	flutter_map, latlong2, FlutterMap, MapOptions, TileLayer, MarkerLayer, Marker, MapController, LatLng, IconButton	5 Hrs
2	**Building Information**	Keep	InkWell / GestureDetector, showModalBottomSheet or side pane, ListView, Column, Row, Image, ExpansionTile, FutureBuilder / StreamBuilder, Building data model. Selecting OpenStreetMap/flutter_map marker or building opens its information pane	3 Hrs
3	**Search**	Keep	SearchBar or TextField, TextEditingController, ListView.builder, FutureBuilder, setState(), filters, result cards, map camera update through MapController	3-5 Hrs
4	**Chatbot Assistant**	Keep	ListView.builder, TextField, TextEditingController, ScrollController,	5 Hrs

			IconButton, Future, loading indicator, message model, API client / http package	
5	**Admin Dashboard / Campus Data CRUD**	Keep	Form, TextFormField, ListView.builder, FutureBuilder, setState(), Supabase select/insert/update/delete, validation, navigation	5 Hrs

Total of 21 - 23 Working Hours

4. **Stretch Goals**
    a. **3D Campus Map -** Upgrade the working 2D map with MapTiler/MapLibre vector styles and building extrusion after the map, search, building information, and Supabase data are stable. This requires maplibre_gl, MapLibreMap, MapLibreMapController, CameraPosition, vector tile styles, extrusion layers, and a 2D/3D toggle.
    b. **Custom CAD-based Campus Models -** Uses accurately designed building models instead of generic 3D map tiles, implemented in Flutter using model_viewer_plus or flutter_3d_controller, StatefulWidget, GestureDetector, Stack, FutureBuilder, and 3D asset loading to display and interact with detailed campus building models.
    c. **Community Reviews and Ratings -** Allows students and other campus users to share ratings, feedback, and firsthand experiences about buildings, facilities, and services, implemented using Form, TextFormField, flutter_rating_bar, ListView.builder, StreamBuilder, user authentication, and database CRUD operations to submit, store, and display reviews.
5. **How My App Saves Data**
    a. **Shared data question -** ligHAU is a campus map, which means all users should see the same shared campus data, such as buildings, classrooms, facilities, parking

areas, and other location information. For the MVP, the only user-specific persistent data will be administrator account/profile information; personal map preferences will not be stored.

b. **Record volume -** Around 100-300 records which initially covers buildings, classrooms, and other campus facilities. In a normal week this would only change slightly since most campus information is already permanent, with administrators mainly updating a few records as required.

c. **My choice: supabase_flutter**

d. **Why this one and not the others? -** I will use Supabase with supabase_flutter to store buildings, rooms, facilities, user accounts, and other shared campus data because all users need to see the same updated information. I chose it over local storage because the app may hold hundreds of records and needs centralized querying and synchronization, accepting the added setup and internet dependency in exchange for shared, persistent data.

e. **What I save, concretely -** I will save campus data in Supabase using a Building class stored in a buildings table with fields such as id, name, description, latitude, longitude, category, floors, and updated_at. Related Room and Facility classes will be stored in rooms and facilities tables using building_id as a foreign key, with fields for room/facility name, type, floor, description, and accessibility information.

f. **Have I tried it yet? -** I have not tried the one-hour Supabase spike yet. I plan to test it during the first week of implementation by connecting supabase_flutter, creating one sample table, inserting a record, and reading it back in the app.

6. **One Thing I Want to Add that the Course Didn't Teach -** For the campus map, I will use flutter_map with latlong2 and OpenStreetMap tiles. It works in Flutter Web and Codespaces, so the map can still open and be tested in the browser without a real device. GPS will not be required for the core experience, since users can search and browse the campus manually. While for the chatbot, I will use the Gemini API through google_generative_ai or http. It also works on Flutter Web, so it can be tested in Codespaces without needing a phone. If the AI service is unavailable, the app can still open normally and show a fallback message or predefined campus responses. These are

both core features because they help users ask questions about campus locations and facilities, while the map provides the main interface for locating buildings and facilities.

7. **How My Projects Run When Someone Else Opens It**

- I am keeping the device_preview wrapper: **Yes**, so I can test different screen sizes and layouts while developing in Flutter Web.
- My app runs in a browser with flutter run -d web-server, start to finish, with every screen reachable: **Not yet**, I have not started building the application yet, so the screens, navigation, Supabase integration, campus map, search, and chatbot still need to be implemented
- Anything that needs real hardware degrades to sample data instead of crashing: **Not applicable for the core app**, since GPS is not required. Users can search and navigate the map manually if location services are unavailable.
- API keys, passwords, or personal data: I will need **Supabase configuration and a Gemini API key**. Local configuration will be stored in a git-ignored .env file, deployment values will use repository secrets, and no real passwords or personal user data will be committed to the public repository.

8. **Data the App Remembers**

Thing	Fields	Where it is saved
Building	id, name, description, latitude, longitude, category, floors, updated_at	Supabase buildings table
Room	id, building_id, name, room_number, type, floor, description, accessibility, updated_at	Supabase rooms table
Facility	id, building_id, name, type, floor, description, accessibility, latitude, longitude, updated_at	Supabase facilities table
User Profile	id, email, role, created_at	Supabase profiles table; authentication handled by Supabase Auth

Chatbot Knowledge	id, question, answer, building_id, facility_id, updated_at	Supabase chatbot_knowledge table

9. **Screens**
    a. Login Screen - Administrator authentication and Continue as Guest.
    b. Campus Map Screen & Utility Panes - 2D map, search, building/facility information, and chatbot.
    c. Admin Dashboard Screen - Add, edit, and remove building, room, facility, and chatbot-knowledge records.
10. **Risk (Revised)**

The risk I named last time: The Admin Dashboard and data synchronization are still a risk, but the UI side feels smaller after Modules 4 and 5 because I now understand stateful widgets, forms, lists, and navigation better. The bigger remaining challenge is authentication and making sure changes in Supabase immediately appear in the campus map, search, building information, and chatbot.

First step: Create a small Supabase test with one buildings table and verify that Flutter can add, read, and update one building record. By September 22, 2026.

A new risk I did not see before: Map integration and campus data accuracy are now risks because the project depends on OpenStreetMap/flutter_map for the core map, while detailed HAU-specific rooms and facilities will still need to be collected and maintained manually.

First step: Build a simple flutter_map test showing the HAU area with at least one clickable marker before adding search or detailed facility data. By September 25, 2026.

Section	Prelim Said	Now Says	Why It Changed

Core Features	4 core features with general descriptions	2D map, building information, search, admin CRUD, and chatbot are MVP with 3D map moved to stretch.	Building the StatefulWidget attack activity showed me that normal UI state is manageable, while the map still requires packages and APIs I have not implemented before. I therefore kept the working 2D map in the MVP and removed 3D from the critical path.
Admin Functionality	Admin Dashboard was planned as a screen but its implementation was not detailed	Admin Dashboard now explicitly performs CRUD operations on buildings, rooms, facilities, and chatbot knowledge through Supabase	The Form/TextField and scrolling-list activities showed me what widgets are required for data-entry screens, so I can now define the admin functionality more precisely.
Persistence	Storage method was not finalized	Supabase with supabase_flutter, using shared tables for buildings, rooms, facilities, users, and chatbot knowledge	After working with models, forms, lists, and navigation, I could identify which information actually needs to persist. Since every student must receive the same campus updates, browser/local-only storage would not satisfy the requirement.

Maps / Extensions	Interactive 2D/3D map without a finalized implementation	flutter_map + OpenStreetMap for the MVP while MapTiler/MapLibre 3D is moved to a stretch goal	Researching the actual Flutter packages showed that 3D requires a second mapping stack, vector styles, camera control, and extrusion configuration, while flutter_map is enough to complete the required navigation workflow.
Risks	Admin Dashboard and authentication were the main risk	Supabase synchronization remains a risk, and accurate HAU map/facility data is a new risk	Stateful widgets, lists, forms, and navigation made the interface less uncertain, but planning the real database and map exposed synchronization and campus-data problems that were not visible in the prelim proposal.
Stretch Goals	Custom CAD campus models and community reviews	Both remain stretch goals and the proposed 3D campus map from prelim is now added as a stretch goal	The 3D map would require additional MapTiler/MapLibre setup, vector styling, camera controls, and building extrusion. Since a 2D map already supports the main campus-navigation purpose, 3D is no longer required for the MVP and would add unnecessary development time.

Everything Else *(Purpose, Audience, Screens, Routes)*	Original ligHAU concept and screen structure	Mostly unchanged	Building Modules 4 and 5 confirmed that the original screen structure and navigation flow are achievable, so there is no strong reason to redesign them.